Architecture Compliance Checklist

Purpose

This checklist exists to preserve the architectural integrity of Shio Runtime.

Every significant feature, Runtime modification or architectural change should be evaluated against these questions before implementation begins.

The objective is not restricting development.

The objective is ensuring that new functionality strengthens the architecture instead of gradually eroding it.

Responsibility

Does the new feature belong to exactly one Runtime?

A responsibility should never be shared between multiple Runtime systems.

If ownership is unclear, the architectural boundary should be reconsidered.

Does the feature introduce a new responsibility?

If yes, determine whether:

- an existing Runtime should own it;
- a new Runtime should be introduced.

Never overload an existing Runtime with unrelated responsibilities.

Runtime Boundaries

Does the implementation cross Runtime boundaries?

If Runtime A begins modifying Runtime B directly, the architecture has likely been violated.

Communication should occur through:

- Character State;
- Event Bus;
- Public Contracts.

Does one Runtime depend on another Runtime's implementation?

Runtime systems may depend on contracts.

They must never depend on implementation details.

Character State

Is Character State being used correctly?

Character State represents the current condition of the character.

It should never be used to represent transient events.

Should this information become Character State?

If the answer is "yes", determine:

- which Runtime owns it;
- which Runtime observes it.

Event Bus

Is this information an Event instead of State?

Ask:

> Does this describe something that happened?

If yes, it is probably an Event.

Is the Event immutable?

Events should describe completed facts.

They should never describe future intentions.

Action Runtime

Does this feature modify the outside world?

If yes:

It must pass through Action Runtime.

No exceptions.

Does the implementation bypass Action Runtime?

If yes:

The architecture has been violated.

Experience Layer

Is execution being confused with experience?

Executing an action and reacting to it are independent responsibilities.

The Runtime should preserve this distinction.

Should the character react?

If yes:

The reaction belongs to Experience Layer.

Memory Layer

Is this information worth remembering?

Not every interaction deserves long-term memory.

Determine whether the information contributes to future continuity.

Is this an implementation detail or a character experience?

Memory should preserve experiences.

Not internal Runtime mechanics.

External Adapters

Is the Runtime communicating directly with a technology?

If yes:

An Adapter is probably missing.

Runtime systems communicate with architectural concepts.

Adapters communicate with technologies.

Replaceability

Could this technology be replaced tomorrow?

If replacing:

- the LLM;
- the TTS engine;
- the ASR engine;
- the storage provider;
- the rendering engine;

would require architectural redesign, the abstraction is insufficient.

Runtime Contract

Does the new Runtime satisfy the Runtime Contract?

A Runtime should:

- own one responsibility;
- own its lifecycle;
- own its resources;
- publish events;
- observe Character State;
- remain independently understandable.

Simplicity

Is the simplest Runtime making the decision?

Avoid unnecessary reasoning.

Prefer deterministic logic whenever possible.

Only escalate complexity when simpler layers cannot make the correct decision.

Character

Does this make Shio feel more believable?

This question should always be answered before implementation.

If a feature increases complexity but does not improve the character, reconsider whether it belongs in the Runtime.

Final Question

Before merging any significant architectural change, ask one final question:

> **If this feature did not exist today, where would it naturally belong within the current architecture?**

If the answer requires changing multiple Runtime responsibilities, introducing hidden dependencies or violating existing principles, the design should be reconsidered before implementation.

Architecture should remain coherent as the project evolves.

New functionality should strengthen the Runtime rather than reshape it.

Guiding Principle

Every architectural decision should reinforce the same foundational idea:

> **Shio Runtime is designed around responsibilities, not technologies.**

Whenever uncertainty exists, return to this principle.

If the design respects responsibilities, the architecture will remain healthy even as technologies evolve.